LRS Zadanie 1 – dokumentácia

1. Prehľad riešenia

Zadanie sme riešili ako ROS 2 (Jazzy) workspace s dvoma balíkmi v C++17. Balík `lrs_planning` plánuje trasu v hale bez kolízií (A1.1) a `lrs_control` ovláda dron cez MAVROS, v hale (A1.2) aj vonku (A1.3). Všetko sme testovali v Gazebo Harmonic s ArduPilot SITL, logiku misií sme si navyše overovali v jednoduchom offline simulátore bez ROS. Kód je v repozitári `D0mc1c1/LRS_Maslen_Kormanikova` vo vetve `devel`.

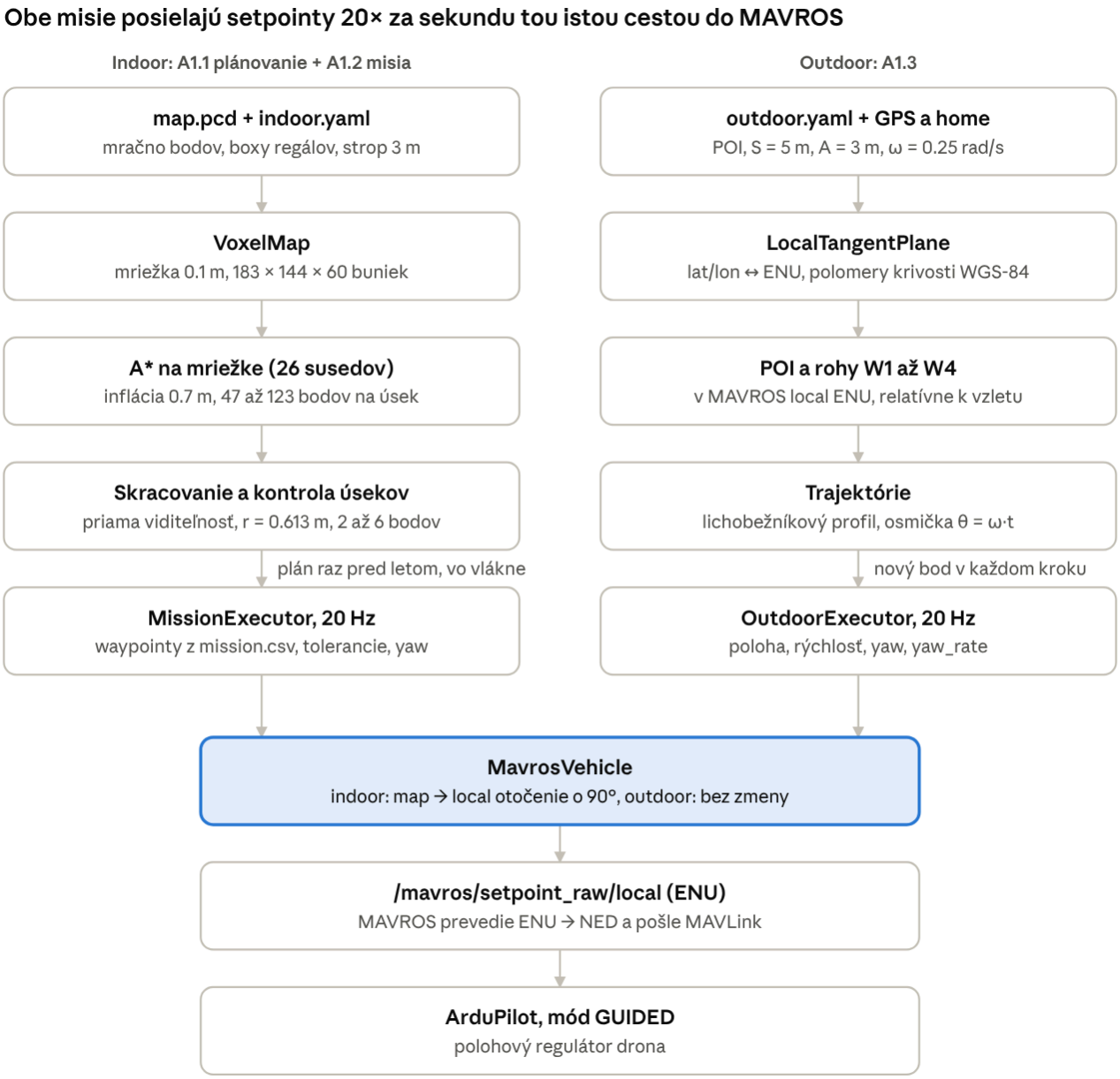
Pri návrhu sme sa držali troch vecí:

- Plánovač je samostatná knižnica, ktorú `lrs_control` volá priamo. Na začiatku misie naplánuje všetky úseky naraz v samostatnom vlákne, takže riadiaca slučka na neho nemusí čakať.
- Stavové automaty (`MissionExecutor` pre indoor a `OutdoorExecutor` pre outdoor) nevedia nič o ROS, s dronom pracujú len cez rozhranie `Vehicle`. Pre ROS ho implementuje `MavrosVehicle`, pre testy `SimVehicle`, vďaka čomu sme vedeli každý scenár otestovať za pár sekúnd bez simulátora.
- Callbacky sú v `MultiThreadedExecutor` rozdelené do troch skupín: riadiaci časovač 20 Hz spolu so službou `~/abort`, odbery tém z MAVROS (zapisujú pod mutexom) a odpovede MAVROS služieb, ktoré sa spracúvajú asynchrónne, aby na ne riadenie nečakalo.

Program	Balík	Účel
`plan_cli`	lrs_planning	A1.1: plánovanie z príkazového riadku
`plan_viz`	lrs_control	A1.1: mapa a plán v RViz bez simulátora
`indoor_mission`	lrs_control	A1.2: misia z CSV súboru
`mission_sim`	lrs_control	A1.2: offline test stavového automatu
`gps_info`	lrs_control	A1.3: výpis GPS, home, kompasu a prevodov
`geo_test`	lrs_control	A1.3: presnosť prevodu LLA↔ENU
`outdoor_mission`	lrs_control	A1.3: outdoor misia
`outdoor_sim`	lrs_control	A1.3: offline test, 14 scenárov

Program	Balík	Účel
`record_outdoor.sh`, `plot_outdoor.py`	lrs_control	A1.3: rosbag a grafy

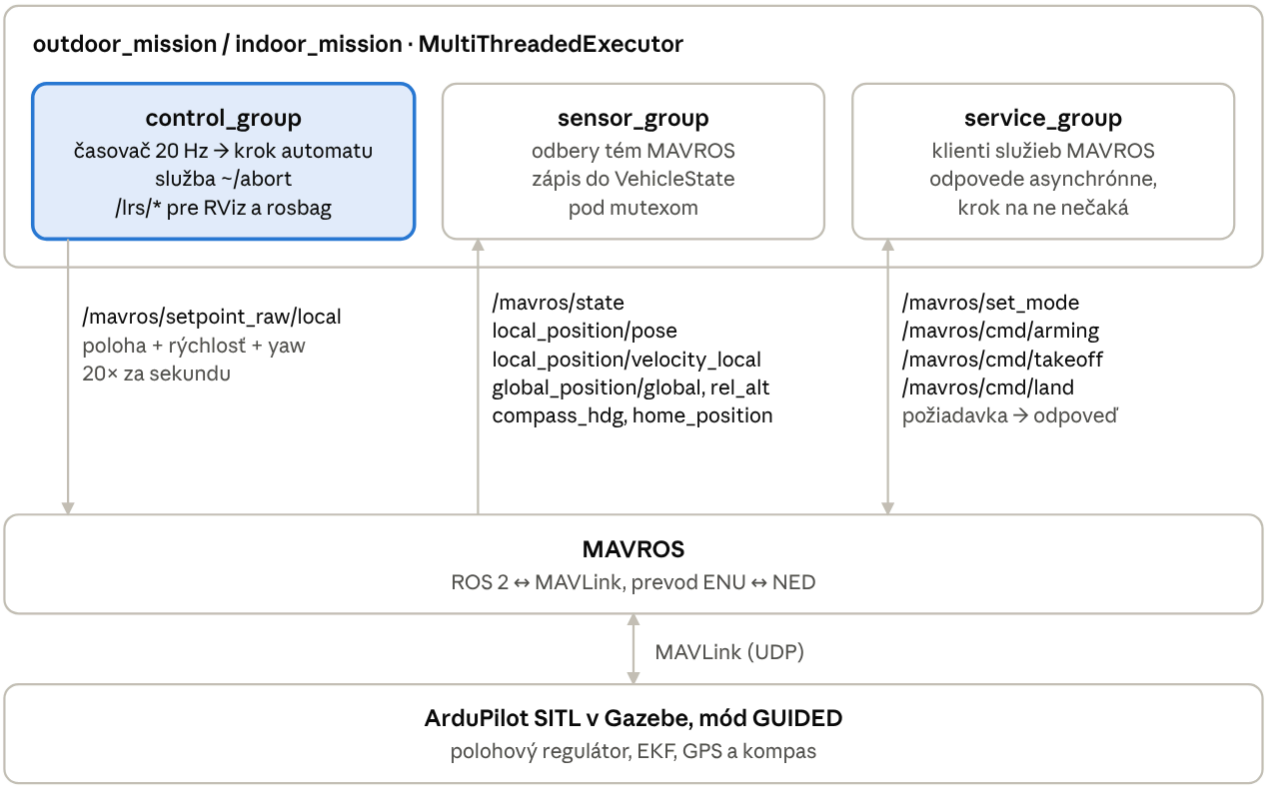
2. Diagramy riešenia



cesta dát · indoor a outdoor

Indoor plánovač sa spustí iba raz pred letom, outdoor trajektórie sa naopak počítajú priebežne v každom kroku počas letu. Od `MavrosVehicle` ďalej je to pre obe misie rovnaké.

Riadenie, dáta z MAVROS a služby bežia v troch oddelených vláknach



ROS riadenie · uzol, skupiny callbackov, MAVROS

Automat beží len v `control_group`, takže v ňom zámky nepotrebujeme. Mutex chráni iba spoločný `VehicleState`. Odpovede služieb chodia do `service_group`, a preto krok automatu nikdy nečaká na MAVROS.

3. A1.1 Mapa a plánovanie

Pre celú skúšobnú misiu (7 úsekov) plánovač nájde cestu bez kolízie a po úprave z nej ostane 21 bodov namiesto 484. Bez simulátora (Release build) to trvá asi 10 s, na našom PC so spusteným Gazebom a SITL asi 35 s.

Reprezentácia mapy

Mračno bodov `map.pcd` načítame do hustej voxelovej mriežky s bunkou 0.1 m. To je 183 × 144 × 60 = 1.58 mil. buniek, pri 1 bajte na bunku asi 1.6 MB. Bunka je obsadená, ak do nej padne aspoň jeden bod.

Veľkosť bunky 0.1 m sme zvolili podľa najužšieho miesta misie, ktorým je ulička pri regáloch; waypoint (2.81, 8.15) je od regálu len 0.81 m. Chyba diskretizácie je najviac polovica uhlopriečky bunky (8.7 cm) a do rezervy sa zmestí. S bunkou 0.2 m by sa ulička zavrela, pri 0.05 m by bolo buniek 8× viac a A* by bežal desiatky sekúnd.

Ako alternatívu sme zvažovali octree (OctoMap), ktorý šetrí pamäť pri veľkých riedkych mapách. Pridal by však ďalšiu závislosť a dotaz je pomalší, pričom hala 18 × 14 × 6 m sa do hustej mriežky zmestí bez problémov.

Výhodou mriežky je dotaz na obsadenosť v O(1), jednoduchá implementácia a deterministické správanie. Nevýhodou je, že pamäť rastie s objemom a rozlíšenie je v celej hale rovnaké.

Inflácia prekážok a regály

Prekážky zväčšujeme o polomer 0.7 m: bod je neprípustný, ak je bližšie ako 0.7 m k stredu obsadenej bunky. Nepočítame to dopredu pre celú mapu, ale až pri dotaze (guľa okolo bodu).

Hodnota 0.7 m je polomer drona (asi 0.3 m) plus tolerancia prechodu medzibodom 0.4 m, o ktorú môže dron pri prepnutí na ďalší bod skrátiť roh. S polomerom 0.5 m viedla cesta medzerou medzi stenou a regálom širokou 1.07 m a voľný koridor mal len 7 cm. Viac ako 0.8 m sa nastaviť nedá, lebo potom by waypoint pri regáli ležal v prekážke.

Regály sú v mračne bodov len ako riedke body a A* by medzi nimi našiel dieru. Preto ich v plánovači pridávame ako dva plné kvádre (obal kolíznych boxov z `models/fei_lrs_racks/model.sdf`), ktoré sa tiež zväčšujú o 0.7 m.

Nad regálmi v mape chýbajú body, takže A* bez obmedzenia naplánoval let ponad regál vo výške 4.8 m. Preto sme pridali strop: plný box nad celou mapou od výšky 3.0 + 0.7 m, takže stred drona sa dostane najvyššie do 3.0 m.

Výhodou je, že bezpečnosť nastavuje jediný parameter a cez regály ani strop sa nedá „prekĺznúť“. Nevýhodou je pomalší A* (dotaz prehľadáva až 15^3 buniek), ručne zadané boxy regálov v `indoor.yaml` a to, že veľký polomer zatvára úzke priechody.

Algoritmus

Použili sme A* na mriežke s 26 susedmi. Cena hrany je jej dĺžka v metroch (1, $\sqrt{2}$ alebo $\sqrt{3}$ krát 0.1 m) a heuristika je euklidovská vzdialenosť k cieľu, tiež v metroch.

Heuristika nikdy neprecení skutočnú cenu (je prípustná aj konzistentná), takže A* nájde najkratšiu cestu v grafe mriežky. V spojitom priestore to najkratšia cesta nie je, lebo sa dá ísť len v 26 smeroch. V pôvodnom kóde sme opravili tri chyby: heuristika a cena boli v rôznych jednotkách (bunky a metre), tabuľka susedov bola zlá a limit 100 000 iterácií bol pri 1.6 mil. buniek malý (zvýšili sme ho na 2 mil.).

Na našom PC pri bežiacom Gazebe trvá jeden úsek 0.55 až 14.1 s, všetkých 7 úsekov skúšobnej misie spolu 35.1 s. Bez simulátora je to 0.17 až 4.7 s na úsek. Plánuje sa len raz na začiatku misie vo vedľajšom vlákne, kým dron stojí na zemi.

Zvažovali sme aj RRT a RRT*, ktoré sú rýchle vo veľkých priestoroch, ale nie sú deterministické a ich cesty sú kostrbaté. Dijkstra by našiel rovnakú cestu, ale bez heuristiky pomalšie, a Theta* by dal priamejšie cesty za cenu zložitejšej implementácie.

A* nájde cestu vždy, keď existuje, na mriežke je optimálny a výsledok je zakaždým rovnaký. Nevýhodou sú sekundy na úsek a cesta, ktorá sa drží smerov mriežky, a preto ju treba upraviť.

Úprava cesty

Cestu skracujeme podľa priamej viditeľnosti: z aktuálneho bodu hľadáme najvzdialenejší bod cesty, ku ktorému vedie voľná úsečka, a body medzi nimi vynecháme. Úsečku kontrolujeme vzorkami každých 5 cm s plným polomerom 0.7 m.

Úsek skúšobnej misie	Body z A*	Po úprave
WP1 → WP2	51	2
WP2 → WP3	47	4
WP3 → WP4	123	6
WP4 → WP5	75	3
WP5 → WP6	67	3
WP6 → WP7	121	3

Po úprave sa každý úsek ešte raz skontroluje. A* overuje len stredy buniek a úsečka medzi susednými stredmi sa môže k prekážke priblížiť o polovicu uhlopriečky bunky, preto kontrola používa polomer 0.7 − 0.087 = 0.613 m. Ak by niektorý úsek neprešiel, plán sa zamietne.

Splajnom cestu nevyhladzujeme. Plynulý pohyb zabezpečí polohový regulátor ArduPilota s jeho limitmi zrýchlenia a cez medzibody dron preletí bez zastavenia (kapitola 4).

Je to jednoduché, bodov je oveľa menej a úseky sú priame. Postup je však hltavý, takže výsledok nemusí byť najkratšia možná cesta, a rohy ostanú ostré.

4. A1.2 Indoor misia

Misiu načítame z CSV súboru a vykonáva ju stavový automat `MissionExecutor`. Skúšobná misia (7 waypointov, úlohy `yaw180`, `landtakeoff`, `yaw90`, `land`) prebehla v Gazebe so SITL celá bez zásahu (kapitola 8), v offline simulácii s A* trvala 102 s.

Rámce

Mapa, CSV aj Gazebo používajú rovnaký rámec „map“, v ktorom os X smeruje na sever (`gazeboXYZToNED = 180 0 0`). MAVROS pracuje v local ENU, kde X je východ, takže `local = R(+90°) · map` , teda `local = (−y, x, z)` . Overili sme to na štarte: bod (13, 7) z mapy MAVROS hlási ako (−7, 13). Prevod robí len `MavrosVehicle` , automat pracuje iba v rámci mapy.

Stavový automat

`WAIT_CONNECTION → PLAN → SET_MODE → ARM → TAKEOFF → ROTATE → NAVIGATE → ARRIVE → TASK_YAW → LAND → GROUND_WAIT → NEXT_WAYPOINT → DONE` , plus `ABORT` . Časovač s frekvenciou 20 Hz zavolá vždy jeden krok automatu a keď je dron vo vzduchu, v každom kroku pošle setpoint. V stave `PLAN` beží A* cez `std::async` , takže automat medzitým beží ďalej. Inak by plánovanie (až 35 s) zablokovalo slučku a rozhodilo časovanie stavov.

Polohový regulátor a príchod

Polohu reguluje ArduPilot v móde GUIDED, náš uzol mu len posiela cieľový bod a rozhoduje, kedy ho dron dosiahol.

Bod	Dosiahnutý, keď	Prečo táto hodnota
`hard` waypoint	vzdialenosť < 0.15 m a rýchlosť < 0.10 m/s nepretržite 1 s	SITL drží polohu na ±3 cm; 0.15 m je jasne „na bode“ a ustáli sa za 1–2 s
`soft` waypoint	vzdialenosť < 0.50 m, bez zastavenia	prelet: dron nebrzdí, len prepne cieľ
medzibod z plánovača	vzdialenosť < 0.40 m, bez zastavenia	menej ako rezerva inflácie (0.7 − 0.3 m polomer drona), takže skrátený roh ostane mimo prekážok

Vzdialenosť meriame v 3D. Na medzibodoch dron nezastavuje, cieľ sa prepne na ďalší bod už 0.4 m pred nimi. Zastaví iba na `hard` waypointe, pred úlohou a v zákrute ostrejšej ako 60° (časť Natočenie).

Natočenie

- Počas letu je yaw nastavený na smer aktuálneho úseku `atan2(Δy, Δx)`. Pri úseku kratšom ako 0.2 m vodorovne (zvislý pohyb) sa yaw nemení. V stave `ROTATE` sa dron pred úsekom najprv otočí na mieste (tolerancia 5°, 0.3 s) a až potom letí. Robí to v troch prípadoch: keď stojí (štart, po `hard` waypointe, po yaw úlohe), keď sa po `soft` waypointe zmení smer o viac ako `turn_stop_deg` = 60° a keď sa cesta v medzibode z plánovača zalomí o viac ako 60°. Miernejšie zákruty preletí bez zastavenia. Bez tohto pravidla dron po soft bode alebo v ostrej zákrute letel bokom, kým sa nos dotočil (offline až 66° počas 1.2 s, v druhej misii až 90°). S ním je odchýlka najviac 26 až 31° na zlomok sekundy a misia trvá asi o 6 s dlhšie.
- Cieľový uhol prepočítame na hodnotu najbližšiu aktuálnemu yaw (`unwrapNear`: cieľ + k·360°). Dron sa tak točí vždy kratšou cestou a pri prechode z +180° na −180° sa neotočí o celé kolo.
- Úloha `yawN` znamená uhol N v rámci mapy (0° = +X mapy, kladný proti smeru hodín), ktorý musí dron držať s toleranciou 5° nepretržite 1 s.

Úlohy `takeoff`, `land`, `landtakeoff`

Vzlet volá službu `cmd/takeoff` a za hotový ho považujeme až vtedy, keď dron dosiahne výšku (z ⩾ cieľ − 0.15 m), nie po uplynutí času. Pri pristátí prepneme do LAND a čakáme na disarm, ktorý ArduPilot spraví sám po dosadnutí. Pri `landtakeoff` sa po disarme počká 3 s a spustí sa znova celý vzlet.

5. A1.3 GPS, kompas a prevod súradníc

Outdoor misia pracuje priamo v MAVROS local ENU bez ďalšej transformácie a všetky ciele sú dané relatívne k miestu vzletu, takže v kóde nie je žiadna pevná súradnica. Lat/lon používame na zadanie POI a rohov a na výpis, kde dron naozaj letel.

GPS pred vzletom

`OutdoorExecutor` dron nearmuje, kým nie je GPS použiteľné aspoň 3 s bez prerušenia. Podmienky sú: stav fixu ⩾ 0, horizontálna presnosť z kovariancie `NavSatFix` najviac 2.0 m a aspoň 6 satelitov. Počet satelitov kontrolujeme len vtedy, keď ho MAVROS publikuje (v SITL nie). Tie 3 s sú tam preto, aby dron nevzlietol hneď na prvom krátkom fixe, ktorý sa ešte neustálil.

Počiatok

Za počiatok berieme home z `/mavros/home_position/home`. Ak do 10 s nepríde, použije sa GPS poloha drona na zemi. Pri prvom zázname (17. 10.) home neprišiel a použil sa tento náhradný počiatok, ktorý je v SITL ten istý bod (kapitola 9). Po oprave QoS sa v zázname z 20. 10. počiatok vzal už z home.

MAVROS local (meraný od EKF originu) a ENU od nášho počiatku sú rovnobežné rámce, môžu byť voči sebe len posunuté. Posun určíme raz na zemi: `posun = local_vzlet − ENU(GPS_vzlet)`.

Prevod LLA ↔ ENU a jeho chyba

Na prevod používame lokálnu dotyčnicovú rovinu v počiatku s polomermi krivosti elipsoidu WGS-84. Pre zemepisnú šírku počiatku φ_0 platí:

```
w = \sqrt{1 - e^2 \sin^2\varphi_0}, \quad R_N = \frac{a(1-e^2)}{w^3}, \quad R_E = \frac{a}{w}
```

```
E = (\lambda - \lambda_0)(R_E + h_0)\cos\varphi_0, \qquad N = (\varphi - \varphi_0)(R_N + h_0), \qquad U = h - h_0
```

Opačný prevod je len inverzia týchto vzťahov. GeographicLib sme nepoužili, aby balík nemal ďalšiu závislosť, a na takú malú plochu misie rovina úplne stačí.

Chybu sme porovnali s presným prevodom cez ECEF (program `geo_test`, šírka 48.15°):

Vzdialenosť od počiatku	Chyba
5 m	0.004 mm
50 m	0.24 mm
100 m	0.96 mm
1000 m	9.8 cm

Naša misia sa vzdiali od miesta vzletu najviac 8.1 m, takže chyba je pod 0.01 mm a oproti šumu GPS (desiatky cm až metre) je zanedbateľná.

Konvencie uhlov

Kde	Konvencia	Prevod
yaw v celom `lrs_control`, `/mavros/local_position/pose`, setpointy	ENU: 0° = východ, kladne proti smeru hodín	–
ArduPilot, MAVLink	NED: 0° = sever, kladne v smere hodín	robí MAVROS (posielame ENU s `FRAME_LOCAL_NED`)
`/mavros/global_position/compass_hdg`	kompas: 0° = sever, kladne v smere hodín	`yaw_ENU = 90° − heading`, zabalené do (−180°, 180°]
úloha `yawN` v indoor CSV	mapa: 0° = +X mapy (sever v Gazebe)	`yaw_local = yaw_map + 90°`

Prevod kompasu sme overili zo záznamu: kompas 0° zodpovedá ENU yaw 90° a kompas 137° zodpovedá ENU −47° (90 − 137 = −47).

Výšky

Veličina	Zdroj	Použitie
`NavSatFix.altitude`	MAVROS, v SITL 193.70 m	len výpis: je to výška nad elipsoidom, o 43.7 m (undulácia geoidu) viac než AMSL 150 m
`rel_alt`	výška nad home od ArduPilota	**kontrola**, vypisuje sa pri každom stave
local z	EKF, MAVROS local	**riadi sa:** setpointy sú `z_vzlet + 5 m`, vzlet je hotový pri `z − z_vzlet ⩾ 4.8 m`

Výškou 5 m myslíme 5 m nad miestom vzletu. V zázname sa obe merania zhodujú na 1 cm, vzlet sa ukončil pri 4.91 m podľa oboch.

6. A1.3 Trajektórie

Dron vzlietne na 5 m, 5 s drží polohu a potom obletí štvorec s nosom otočeným na POI. Nasleduje osmička s nosom v smere letu, návrat nad miesto vzletu a pristátie. Stavy idú v poradí `HOLD → POI_ALIGN → SQUARE → FIG8_ENTRY → FIG8_ALIGN → FIG8 → RETURN → LAND`.

Štvorec s nosom na POI

- POI sa zadáva parametrom, buď ako `poi_lat` / `poi_lon`, alebo ako posun od miesta vzletu. Predvolene je to 5 m na sever, aby vzlet a pristátie neboli pod trasou (miesto vzletu je 2.5 m mimo štvorca).
- Rohy štvorca (S = 5 m) sú POI ± 2.5 m. Prepočítame ich na lat/lon a späť do local ENU, takže sa použijú oba smery prevodu, rovnako ako keby ich zadal človek v lat/lon. Poradie je W1 (SZ) → W2 (SV) → W3 (JV) → W4 (JZ) → W1.
- Po strane sa dron pohybuje podľa časového lichobežníkového profilu (zrýchlenie 0.5 m/s², rýchlosť 1.0 m/s, brzdenie do rohu). V setpointe posielame polohu aj rýchlosť z profilu ako feedforward, takže ArduPilot letí spolu s cieľom a nedobieha ho.
- Yaw počítame v každom kroku (20 Hz) zo skutočnej polohy drona: `yaw = atan2(POI_y - y, POI_x - x)`. Otáčame len yaw vo vodorovnej rovine, gimbal nemáme. Do setpointu ide aj analyticky vypočítaná rýchlosť zmeny tohto smeru ako feedforward:

$$\dot\psi = \frac{d_y v_x - d_x v_y}{d_x^2 + d_y^2}, \qquad (d_x, d_y) = POI - p$$

Smer na POI sa mení najrýchlejšie v strede strany, 2.5 m od POI: 1.0 / 2.5 = 0.4 rad/s, čiže 23°/s. Pred štvorcom sa dron najprv na mieste otočí na POI (5°, 1 s), aby sa na začiatku prudko netočil.

Osmička ako časová trajektória

V každom kroku riadiaceho časovača (20 Hz) vypočítame fázu θ a z nej nový setpoint okolo POI vo výške 5 m:

$$x = A\sin\theta, \qquad y = A\sin\theta\cos\theta, \qquad \theta = \omega\,(t - t_{start})$$

Na začiatku dron preletí z W1 do stredu nad POI, cestou sa plynulo stočí na smer, ktorým osmička začína (θ = 0, čiže 45°), dorovná natočenie a až potom sa spustí čas. Vďaka tomu nevznikne skok z rohu do stredu.

V strede má krivka hneď od začiatku plnú rýchlosť $A\omega\sqrt{2}$, preto ω zvyšujeme z 0 na cieľovú hodnotu počas 3 s a na konci ju rovnako znižujeme (parameter `fig8_ramp`, 0 znamená presne $\theta = \omega t$). V offline teste to znížilo odchýlku na štarte z 0.22 m na 0.01 m.

Parametre osmičky

Parameter	Hodnota	Výpočet
A	3 m (zadanie)	rozmer 6 × 3 m
ω	0.25 rad/s	zvolené
max. rýchlosť (v strede)	1.06 m/s	$A\omega\sqrt{2} = 3 \cdot 0.25 \cdot 1.414$
čas kola	25.1 s + 3 s rozbeh/dobeh	$2\pi/\omega$
max. rýchlosť otáčania	45°/s (pri lalokoch)	$3.17 \cdot \omega$
max. vzdialenosť misie od vzletu	7.91 m (roh W1/W2)	$\sqrt{(2.5^2 + 7.5^2)}$
geofence	15 m, výška 5 m	misia sa zmestí

Zadanie by dovolilo ω až 0.35 (1.48 m/s pri limite 1.5 m/s). My sme zvolili 0.25, teda 1.06 m/s, čo je približne ako na štvorci a ostáva rezerva na vietor a reálny dron. Na reálnom drone by sme začali s $\omega = 0.15$ (0.64 m/s). Ak by $A\omega\sqrt{2}$ prekročilo `fig8_max_speed` (1.5 m/s), misia sa vôbec nespustí.

Natočenie počas osmičky

```
\psi = \operatorname{atan2}(\cos 2\theta,\ \cos\theta), \qquad \dot\psi = \dot\theta\,\frac{\cos\theta \cdot (-2\sin 2\theta) - \cos 2\theta \cdot (-\sin\theta)}{\cos^2\theta + \cos^2 2\theta}
```

Smer počítame z derivácií podľa θ, takže je definovaný aj pri $\dot\theta = 0$ a nikdy nie je neurčený, lebo cos θ a cos 2θ nie sú nikdy naraz nulové. Do setpointu posielame uhol aj ψ ako feedforward. Uhol pred odoslaním unwrapneme k predchádzajúcemu setpointu, takže medzi krokmi sa zmení najviac o 2.3° a skok o 360° nenastane. Stredom dron preletí dvakrát, raz s nosom na 45° a raz na 135°, a tak to má byť.

Výhodou je presné sledovanie vďaka feedforward, plynulý štart a ochrana pred príliš rýchlou osmičkou. Nevýhodou je, že yaw spolu s yaw_rate vyžaduje ArduCopter 4.1+ (inak treba `yaw_rate_ff: false`) a kolo s rozbehom trvá o 3 s dlhšie.

7. Bezpečnosť

Program má dva spôsoby, ako skončiť. Pri aborte prepne dron do bezpečného módu. Pri uvoľnení len prestane riadiť a nič neprikazuje; to sa stane vždy, keď dron riadi niekto iný alebo sa stratí spojenie, aby program nebojoval s pilotom.

Udalosť	Reakcia	Overené
služba `~/abort` alebo chyba misie (timeout, odchýlka od osmičky > 1.5 m)	prepne do `abort_mode` (outdoor LAND, RTL, BRAKE alebo LOITER; indoor vždy LAND), po potvrdení módu uzol skončí	offline `abort`, `abortrtl`, `SIM_ABORT`
pilot prepne mód z RC, disarm vo vzduchu	prestane posielať setpointy, nič neprikazuje, uzol skončí	offline `rc`, `SIM_RC`
MAVROS spadne alebo FCU odpojené (`/mavros/state` > 3 s, poloha > 1 s)	to isté ako pri pilotovi; ArduPilot drží pozíciu v GUIDED, ďalej rozhoduje pilot a failsafe	offline `linkloss`, `SIM_LINK`
GPS nepoužiteľné	nearmuje, čaká a každých 5 s vypíše dôvod	offline `nogps`
misia ďalej ako `geofence_radius` (15 m) alebo osmička rýchlejšia ako 1.5 m/s	nevzlietne	offline `geofence`, `toofast`

Stratu spojenia sa nedá spoľahlivo zistiť z príznaku `connected`, lebo po páde MAVROS ostane na poslednej hodnote. Program preto sleduje, aké staré sú posledné správy. RTL v hale nepoužívame, lebo ArduPilot by najprv vystúpal na `RTL_ALT` (predvolene 15 m).

Pred letom na reálnom drone treba mať na pamäti, že misia ide najviac 7.9 m od miesta vzletu a 5 m vysoko, takže geofence na drone musí byť väčší. POI je zadané posunom od miesta vzletu, abort máme overený v simulácii a kompas zo záznamu (kapitola 5).

8. Výsledky zo simulácie

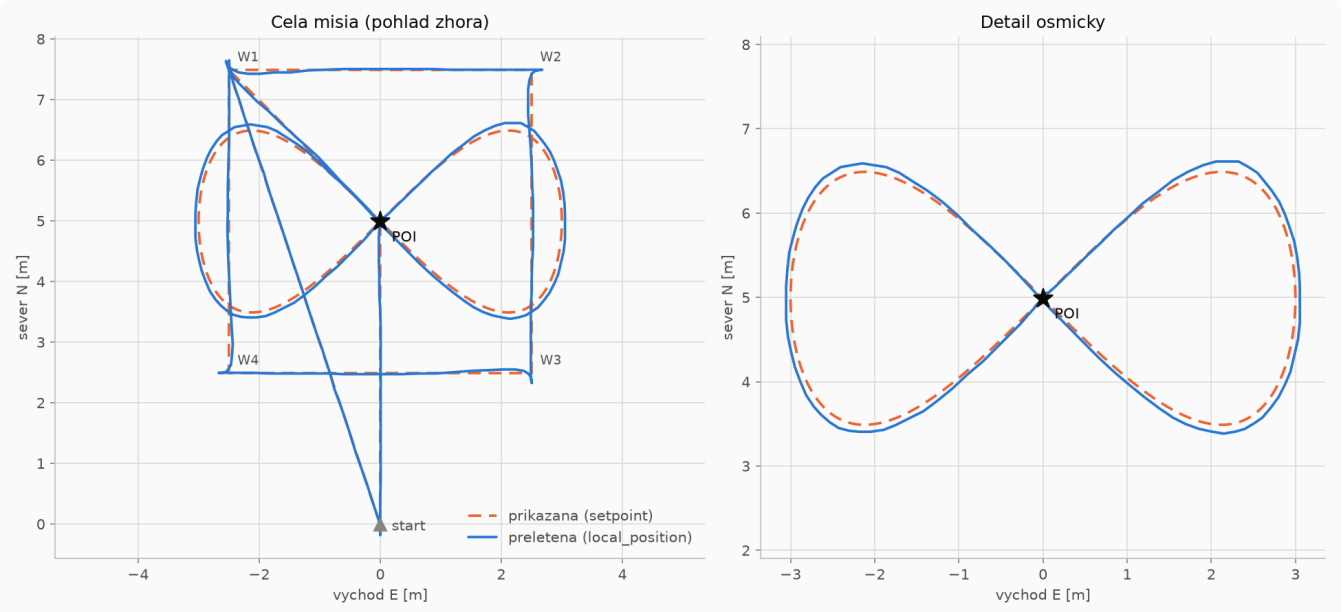
Outdoor misia

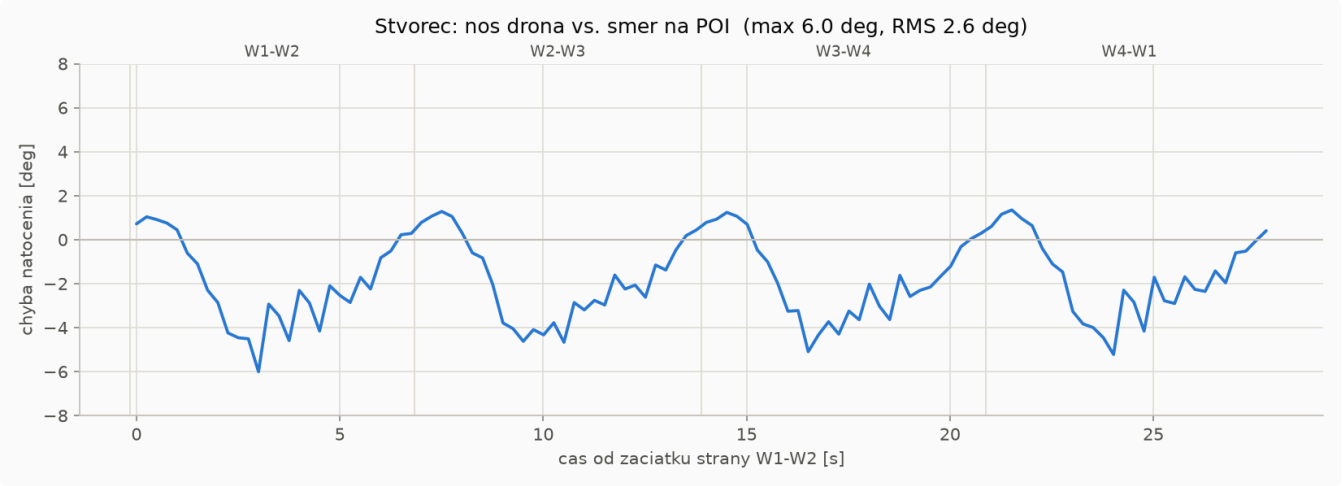
Outdoor misia v Gazebe s ArduPilot SITL prebehla celá bez zásahu, od vzletu po disarm za 108 s, a dron pristál 0.00 m od miesta vzletu. Hodnoty sme vyhodnotili skriptom `plot_outdoor.py` z rosbagu `outdoor_run2` (20. 10. 2026, je v priečinku `bags/` v

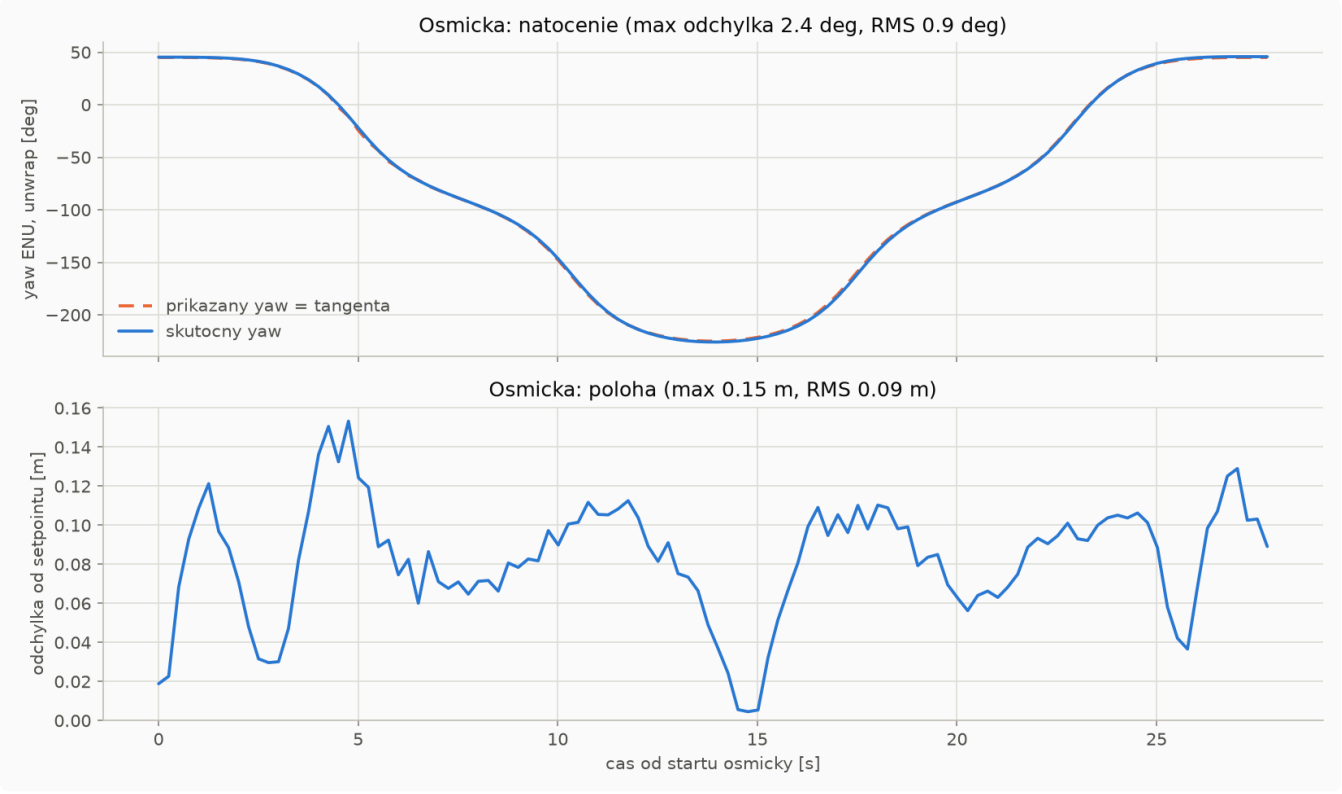
repozitári). Prvý let `outdoor_run1` (17. 10.) vyšiel takmer rovnako, rozdiely boli do 2 cm a 0.8°.

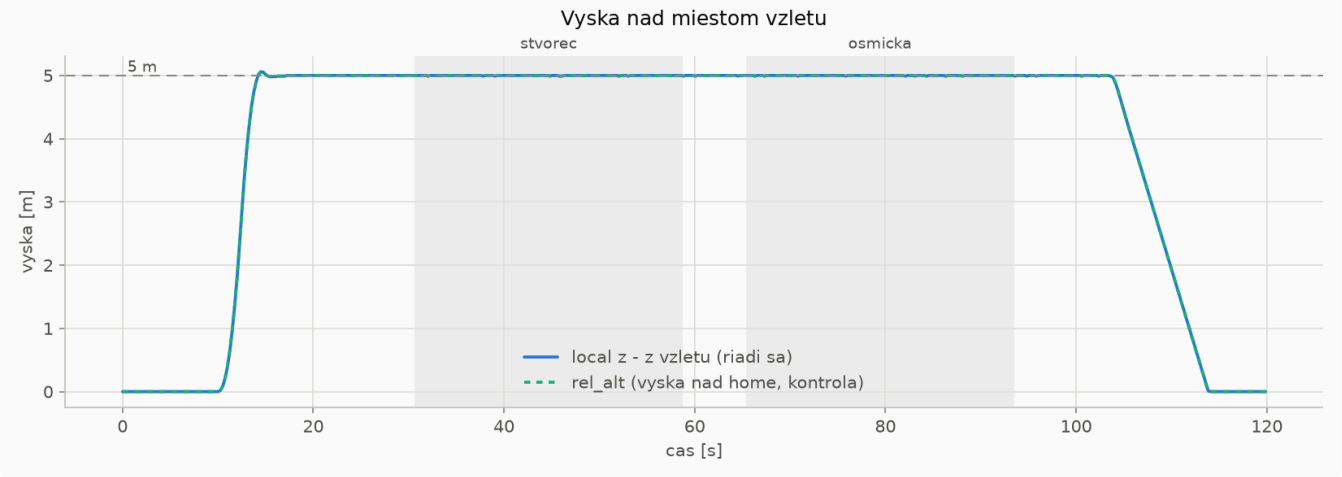
Meranie	Výsledok	Požiadavka
vzlet potvrdený / max. výška	4.91 m / 5.06 m	5 m nad miestom vzletu
držanie pred štvorcom	smerodajná odchýlka výšky 0.7 cm	stabilne
štvorec: nos vs. smer na POI	max 6.0°, RMS 2.6°	„pár stupňov“
štvorec: dron pri prepnutí rohu	0.16–0.17 m od rohu	tolerancia 0.3 m
osmička: poloha vs. setpoint v tom istom čase	max 0.15 m, RMS 0.09 m	tvar zodpovedá
osmička: nos vs. smer dráhy	max 2.4°, RMS 0.9°	bez točenia a skôkov
osmička: rozmery od POI	E −3.05 až 3.05 m, N −1.60 až 1.62 m	±3.0 m, ±1.5 m
max. vzdialenosť od miesta vzletu	8.06 m	plán 7.91 m, geofence 15 m

Na lalokoch je osmička o 5 až 12 cm väčšia ako teoretická, inak sa preletená a prikázaná trasa prekrývajú.




Stvorec: nos drona vs. smer na POI (max 6.0 deg, RMS 2.6 deg)
W1-W2
W2-W3
W3-W4
W4-W1
chyba natocenia [deg]
cas od zaciatku strany W1-W2 [s]


Osmicka: natocenie (max odchylka 2.4 deg, RMS 0.9 deg)
yaw ENU, unwrap [deg]
prikazany yaw = tangenta
skutocny yaw
Osmicka: poloha (max 0.15 m, RMS 0.09 m)
odchylka od setpointu [m]
cas od startu osmicky [s]

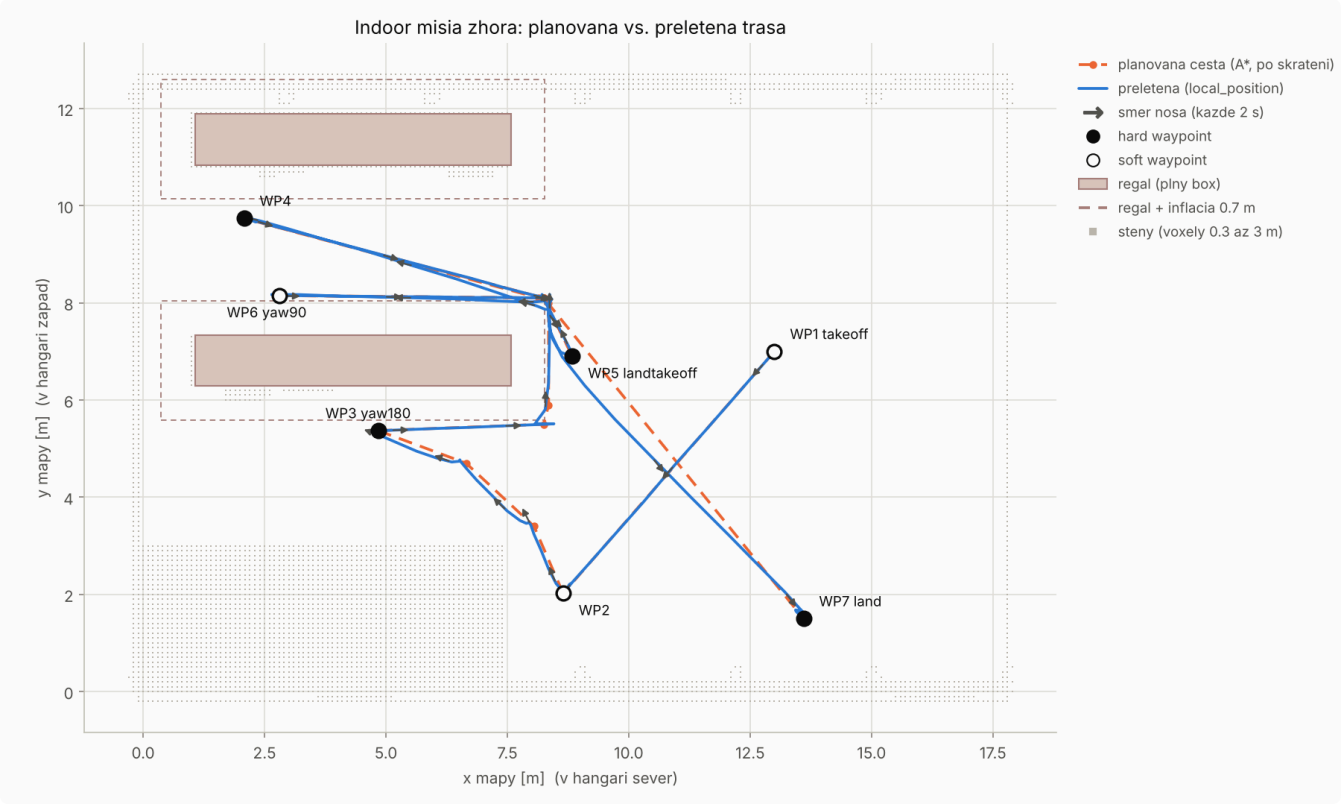

Vyska nad miestom vzletu
stvorec
osmicka
5 m
vyska [m]
local z - z vzletu (riadi sa)
rel_alt (vyska nad home, kontrola)
cas [s]

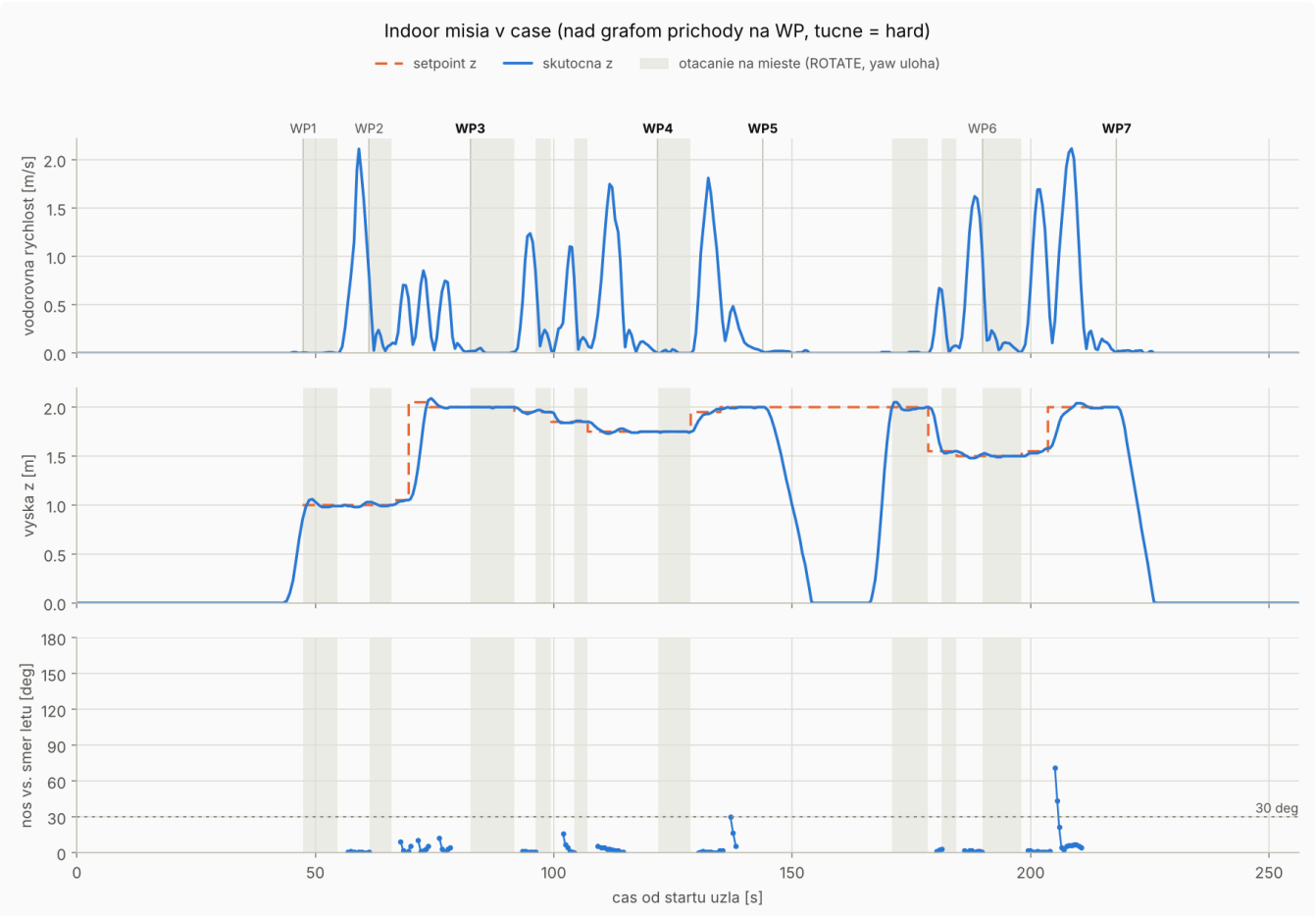
Indoor misia

Skúšobná misia `exam_example.csv` prebehla v Gazebe s ArduPilot SITL celá bez zásahu, všetkých 7 waypointov až po `LAND -> DONE`. Od štartu uzla to trvalo 230 s, z toho 35 s plánovanie na zemi. Hodnoty sú z rosbagu `indoor_run1` (19. 10. 2026), ktorý sme vyhodnotili skriptom `plot_indoor.py`, a z logu misie.

Meranie	Výsledok	Požiadavka
hard waypointy WP3, WP4, WP5, WP7	chyba 0.057 až 0.079 m, rýchlosť pri potvrdení najviac 0.06 m/s	zastaviť a ustáliť sa (0.15 m, 0.10 m/s, 1 s)
soft waypointy WP1, WP2, WP6	chyba 0.15 až 0.40 m pri rýchlosti 0.65 až 2.07 m/s	prelet bez zastavenia
úlohy	yaw180 179.9°, yaw90 89.4°, oba držané 1 s; landtakeoff: disarm, 3 s, znova GUIDED, arm a vzlet	všetky v poradí
odchýlka od naplánovanej cesty počas letu	max 0.55 m, RMS 0.09 m	let po ceste z A1.1
najmenšia vzdialenosť stredu drona od regálu	0.70 m	inflácia 0.70 m, regálmi nepreletí
najmenšia vzdialenosť od iných prekážok (povrch voxelu)	0.56 m	plán zaručuje 0.56 m (0.613 m od stredu voxelu)
nos vs. smer letu (pri rýchlosti nad 0.3 m/s, 49 s letu)	priemer 4.5°, 95 % hodnôt pod 17°, nad 30° spolu 1.5 s	nos v smere letu
otočenia na mieste	9, z toho 3 v zákrutách medzi bodmi (75°, 63°, 64°)	pred úsekom, nie počas neho

Viac ako 30° od smeru letu bol nos len v dvoch zákrutách so zalomením 49° a 51°, ktoré sú pod prahom `turn_stop_deg` = 60°. Tieto zákruty dron preletel a nos sa dotáčal menej ako sekundu (najviac 71°). Poloha sa v bagu ukladala len s frekvenciou 1.9 Hz, takže smer letu počítaný z rozdielov polohy je dosť hrubý. Na dlhších úsekoch ArduPilot zrýchlil až na 2.1 m/s, preto soft body preletel rýchlo.


Indoor misia zhora: planovana vs. preletena trasa
planovana cesta (A*, po skrateni)
preletena (local_position)
smer nosa (kazde 2 s)
hard waypoint
soft waypoint
regal (plny box)
regal + inflacia 0.7 m
steny (voxely 0.3 az 3 m)
WP4
WP6 yaw90
WP1 takeoff
WP5 landtakeoff
WP3 yaw180
WP2
WP7 land
y mapy [m] (v hangari zapad)
x mapy [m] (v hangari sever)


Indoor misia v case (nad grafom prichody na WP, tucne = hard)
setpoint z
skutocna z
otacanie na mieste (ROTATE, yaw uloha)
WP1
WP2
WP3
WP4
WP5
WP6
WP7
vodorovna rychlost [m/s]
vyska z [m]
nos vs. smer letu [deg]
30 deg
cas od startu uzla [s]

9. Známe obmedzenia a testy

Obmedzenia

1. Poloha zo SITL chodí len s frekvenciou 4 Hz (predvolená rýchlosť streamov ArduPilota). Živá metrika osmičky v logu misie (max 0.33 m a 13.7°) preto vychádza horšie, lebo porovnáva až 250 ms starú polohu s aktuálnym setpointom. Po časovom zarovnaní v rosbagu je to 0.15 m a 2.4°. Aj smer na POI sa z polohy aktualizuje len 4× za sekundu a medzitým ho dopĺňa feedforward. Rýchlosť streamu by sa dala zvýšiť cez `/mavros/set_stream_rate`, to sme zatiaľ neskúšali.
2. V prvom lete (17. 10.) neprišiel home. MAVROS ho posiela len pri zmene ako „latched" správu (reliable, transient_local), ale náš odber bol best-effort a volatile. Použil sa preto náhradný počiatok z GPS na zemi, čo je v SITL ten istý bod. Po zmene QoS odberu na reliable + transient_local prišiel home v lete 20. 10. hneď po spustení misie, takže je to opravené.
3. Chyba natočenia na POI rastie do 5 až 6° v strede strán, kde sa smer na POI mení najrýchlejšie (23°/s). Zmenšila by sa pri nižšej rýchlosti `speed`.
4. Plánovanie trvá sekundy (pre skúšobnú misiu 35 s na našom PC pri bežiacom Gazebe, bez neho asi 10 s). Beží len raz pred vzletom; na plánovanie počas letu by bol potrebný rýchlejší algoritmus.
5. Regály a strop sú ručne zadané boxy v `indoor.yaml`, takže pre inú halu treba upraviť parametre.
6. Všetko je overené len v simulácii. Na reálnom drone bude poloha z GPS šumieť (desiatky cm), čo sa prejaví na smere k POI. Yaw spolu s yaw_rate vyžaduje ArduCopter 4.1+.
7. Zákruty pod prahom `turn_stop_deg` (60°) dron preletí a nos sa dotáča menej ako sekundu. V Gazebe to boli dve zákruty so zalomením 49° a 51° a nos sa vtedy odchýlil až o 71°. Prah 45° by aj tieto zákruty riešil otočením na mieste, ale za cenu ďalších zastavení. Pri soft bodoch s ostrou zákrutou dron kvôli otočeniu na chvíľu zastaví, no bez ustálenia ako pri hard bode.
8. Presuny v outdoor misii nesledujú smer letu. Natočenie riešime len tam, kde to zadanie A1.3 vyžaduje: na štvorci na POI a na osmičke po dotyčnici. Pri presune na W1 drží dron nos na POI, vo `FIG8_ENTRY` sa yaw plynulo prelína z POI na dotyčnicu a pri `RETURN` ostane posledný yaw z osmičky. Ak by to bolo potrebné, dá sa to vyriešiť rovnako ako indoor, otočením na smer presunu pred jeho začiatkom.

Testy

Logiku misií sme testovali aj offline, bez ROS a Gazeba, so simulovaným dronom `SimVehicle`, ktorý vie simulovať aj GPS, zásah z RC a výpadok spojenia. Príkazy na spustenie sú v `README.md`.

Program	Scenáre	Výsledok
`outdoor_sim`	normal, noff, noramp, fast, toofast, setpoint, latlon, nohome, nogps, geofence, rc, abort, abortrtl, linkloss	14 / 14 OK
`mission_sim`	test_open, skúšobná misia s A*, `SIM_RC`, `SIM_ABORT`, `SIM_LINK` v rôznych fázach	všetky OK
Gazebo + SITL	outdoor (outdoor_run1, outdoor_run2) aj indoor misia (indoor_run1) so záznamom do rosbagu	prebehla celá (kapitola 8)